```
import os
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image

import tensorflow as tf
from tensorflow.keras import layers, models
```

```
/usr/local/lib/python3.12/dist-packages/jax/_src/cloud_tpu_init.py:86: UserWarning: Transparent hugepages are not enabled. T
  warnings.warn(
```

```
import kagglehub

path = kagglehub.dataset_download("andhikawb/fashion-mnist-png")

print("Dataset Path:", path)
```

```
Using Colab cache for faster access to the 'fashion-mnist-png' dataset.
Dataset Path: /kaggle/input/fashion-mnist-png
```

```
train_dir = os.path.join(path, "train")
test_dir = os.path.join(path, "test")

print("Train Path:", train_dir)
print("Test Path:", test_dir)
```

```
Train Path: /kaggle/input/fashion-mnist-png/train
Test Path: /kaggle/input/fashion-mnist-png/test
```

```
class_names = {
    0: "T-shirt/top",
    1: "Trouser",
    2: "Pullover",
    3: "Dress",
    4: "Coat",
    5: "Sandal",
    6: "Shirt",
    7: "Sneaker",
    8: "Bag",
    9: "Ankle boot"
}

print("Total Classes:", len(class_names))
```

```
Total Classes: 10
```

```
def count_images(directory):
    counts = {}
    total = 0

    for label in os.listdir(directory):
        label_path = os.path.join(directory, label)
        if os.path.isdir(label_path):
            num_images = len(os.listdir(label_path))
            counts[label] = num_images
            total += num_images

    return counts, total

train_counts, train_total = count_images(train_dir)
test_counts, test_total = count_images(test_dir)

print("Train Samples:", train_total)
print("Test Samples:", test_total)

print("\nTrain Class Distribution:")
for k, v in train_counts.items():
    print(f"Class {k}: {v}")
```

```
Train Samples: 60000
Test Samples: 10000

Train Class Distribution:
Class 7: 6000
Class 2: 6000
Class 5: 6000
Class 8: 6000
Class 0: 6000
```

```
Class 3: 6000
Class 1: 6000
Class 4: 6000
Class 9: 6000
Class 6: 6000
```

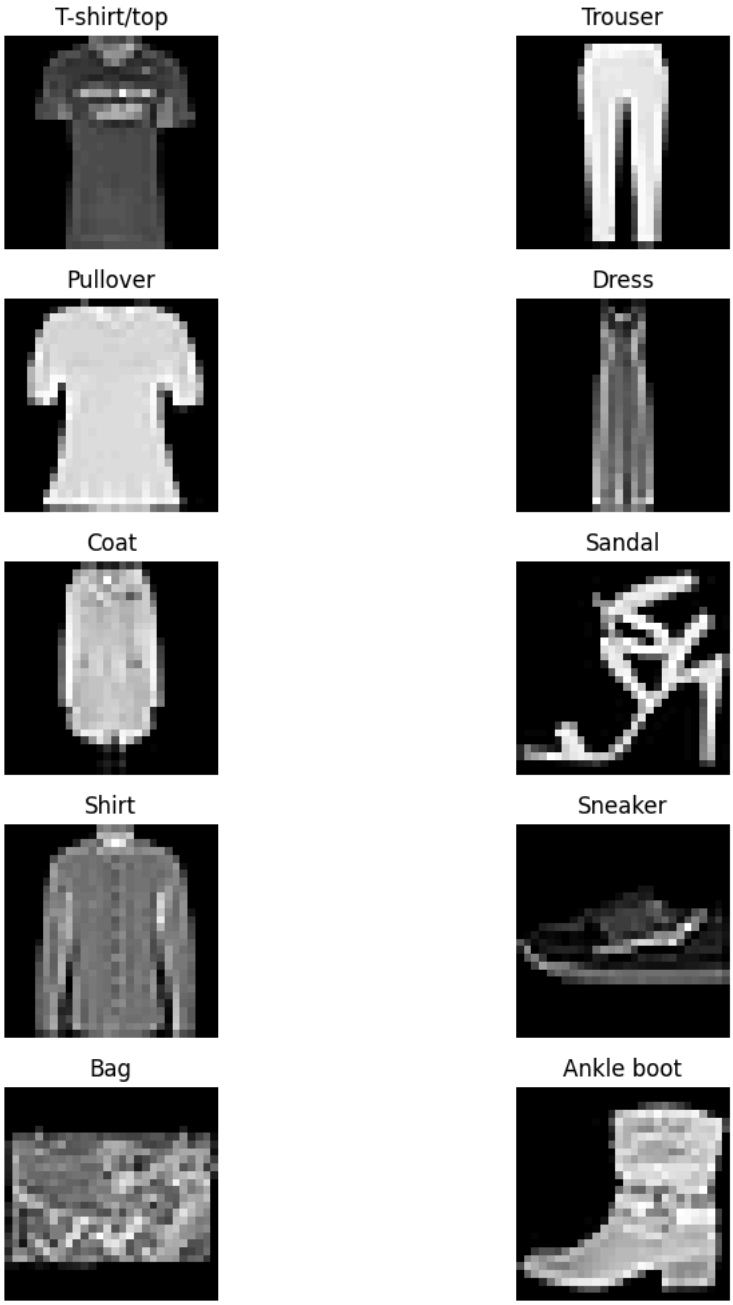
```
plt.figure(figsize=(10,10))

for i in range(10):
    folder = os.path.join(train_dir, str(i))
    img_name = os.listdir(folder)[0]
    img_path = os.path.join(folder, img_name)

    img = Image.open(img_path)

    plt.subplot(5,2,i+1)
    plt.imshow(img, cmap='gray')
    plt.title(class_names[i])
    plt.axis('off')

plt.tight_layout()
plt.show()
```




```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)
```

```
train_data = train_datagen.flow_from_directory(
    train_dir,
    target_size=(28,28),
    color_mode='grayscale',
    class_mode='categorical',
    batch_size=32
)

test_data = test_datagen.flow_from_directory(
    test_dir,
    target_size=(28,28),
    color_mode='grayscale',
    class_mode='categorical',
    batch_size=32
)
```

```
Found 60000 images belonging to 10 classes.
Found 10000 images belonging to 10 classes.
```

```
model = models.Sequential()

# Conv Layer 1
model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D((2,2)))

# Conv Layer 2
model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))

# Conv Layer 3
model.add(layers.Conv2D(64, (3,3), activation='relu'))

# Flatten
model.add(layers.Flatten())

# Dense Layers
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 64)	36,928
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 64)	36,928
dense_1 (Dense)	(None, 10)	650

Total params: 93,322 (364.54 KB)
Trainable params: 93,322 (364.54 KB)
Non-trainable params: 0 (0.00 B)

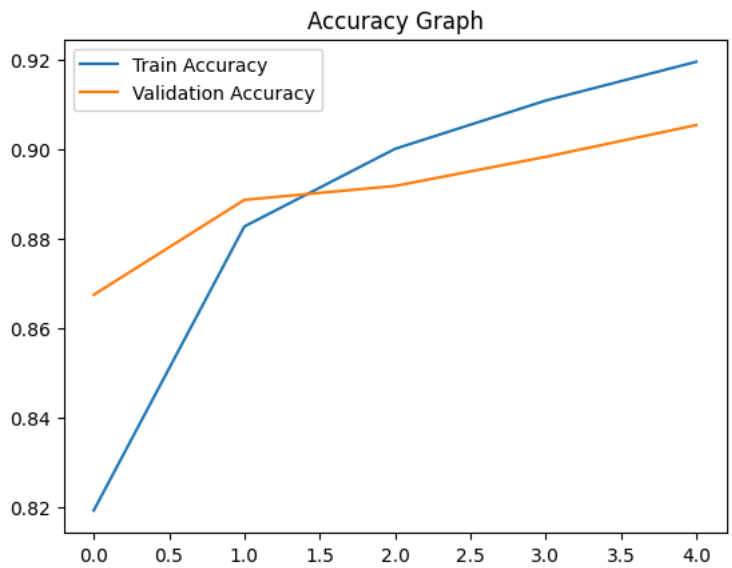
```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

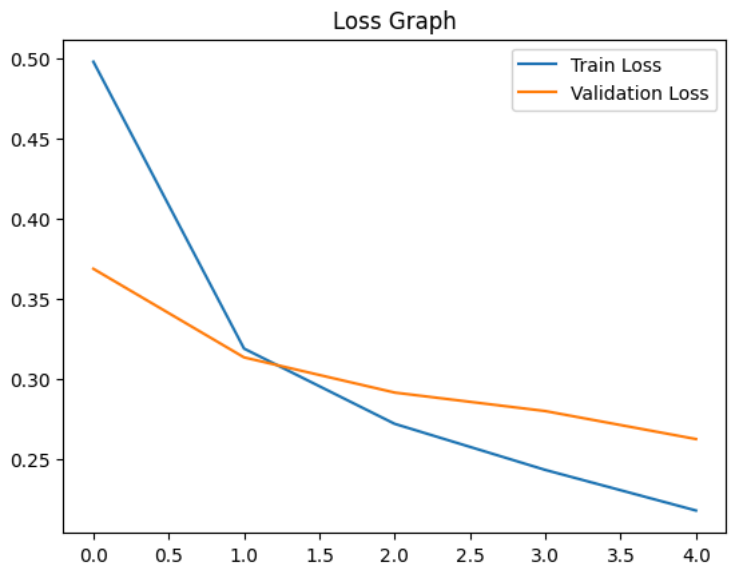
```
history = model.fit(
    train_data,
    epochs=5,
    validation_data=test_data
)
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 328s 174ms/step - accuracy: 0.8193 - loss: 0.4978 - val_accuracy: 0.8675 - val_loss: 0.3688
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 45s 24ms/step - accuracy: 0.8827 - loss: 0.3191 - val_accuracy: 0.8887 - val_loss: 0.3136
Epoch 3/5
```

```
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 44s 24ms/step - accuracy: 0.9001 - loss: 0.2722 - val_accuracy: 0.8918 - val_loss: 0.2917
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 44s 24ms/step - accuracy: 0.9109 - loss: 0.2435 - val_accuracy: 0.8983 - val_loss: 0.2802
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 45s 24ms/step - accuracy: 0.9195 - loss: 0.2182 - val_accuracy: 0.9054 - val_loss: 0.2627
```

```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("Accuracy Graph")
plt.show()
```




```
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Loss Graph")
plt.show()
```